

Making Data Race Detection Efficient

Using Sharing Analysis & Escape Analysis: A Dynamic and Static Approach

Dhruv Roy Talukdar
Rahul Meena

CS636 – Course Project
Indian Institute of Technology Kanpur

August 25, 2026

Contents

1	Introduction	4
1.1	Background and Motivation	4
1.2	The Problem with Traditional Vector-Clock Based Detection	4
1.3	Our Approach: Sharing Analysis	5
2	Preliminaries	5
2.1	FastTrack: HB-Based Race Detection	5
2.1.1	The Happens-Before Relation	5
2.1.2	Vector Clocks and Epochs	6
2.1.3	FastTrack Algorithm	6
2.2	WCP: Predictive Race Detection	7
2.2.1	Motivation for Predictive Detection	7
2.2.2	The WCP Relation	7
2.2.3	WCP State	8
2.2.4	WCP Procedures	8
3	Sharing Analysis: The Optimization Layer	9
3.1	Core Idea	9
3.2	The Hot Word	9
3.3	Shadow Table	10
3.4	Sharing Analysis Applied to FastTrack	10
3.4.1	Owner Epoch Cache	10
3.4.2	Fast Path for FastTrack+SA	11
3.4.3	Slow Path: OWNED-to-SHARED Transition	12
3.4.4	Reclaim: Returning to OWNED State	12
3.5	Sharing Analysis Applied to WCP	13
3.5.1	Key Differences from FT+SA	13
3.5.2	WCP Fast Path	13
3.5.3	WCP+SA Slow Path: Restoring VClock History	14
3.5.4	WCP+SA Reclaim	14
4	Dry Run Examples	15
4.1	Example 1 – Correctness: Same Races Across All Four Algorithms	15
4.1.1	Execution trace	15
4.1.2	Algorithm-by-Algorithm Trace for Correctness Example	16
4.1.3	Correctness Summary	17
4.2	Example 2 – Efficiency: Repeated Lock/Unlock Exposing the SA Advantage	18
4.2.1	Interleaving and Vector Clock Trace	18
4.2.2	Why FT’s Same-Epoch Check Fails Here	19
4.2.3	Slow-Handler Count: All Four Algorithms	19
4.2.4	Efficiency Summary	21

5	Algorithmic Summary and Comparison	21
5.1	Structural Modifications for Sharing Analysis	21
5.1.1	FastTrack	21
5.1.2	WCP	22
6	Static Analysis : Motivation	22
7	The problem and approach	22
7.1	The Approximation Strategy	23
8	Escape Analysis — Background	23
8.1	What Is Escape Analysis?	23
8.2	Classic Optimisations Enabled by Escape Analysis	24
8.3	Why Track Pointers Rather Than Values?	24
9	Existing Solutions and Their Limitations	24
9.1	LLVM CaptureTracking API	25
9.2	Why CaptureTracking Is Insufficient	25
10	Solution: ThreadEscapeAnalysis	26
10.1	Overview	26
10.2	Abstract State Representation	26
10.2.1	State Transitions — Per Instruction	27
10.3	The Control-Flow Graph and Basic Block States	28
10.4	Worklist-Based Fixed-Point Algorithm	29
10.5	Merging States at Join Points	29
10.6	Handling CFG Cycles — Optimistic Merge	30
10.7	Complete Worked Example	31
10.8	Consolidation Phase	32
11	Soundness: A Formal Argument	33
11.1	Property 1: Default-Capture Conservatism	33
11.2	Property 2	33
11.3	Property 3: Soundness of Consolidation	33
12	Performance Results	34
12.1	Benchmark Programs	34
12.1.1	Test 1 — Dense Shared-Variable Access	34
12.1.2	Test 2 — Thread-Private Chunk Access	34
12.2	Results	35
12.3	Analysis	36
12.3.1	Test 1: Fully-Shared Workload	36
12.3.2	Test 2: Thread-Private Workload	37
12.3.3	Cross-Benchmark Summary	37
13	Conclusion	38

A	Appendix: Key Data Structures at a Glance	39
A.1	FastTrack Thread State	39
A.2	WCP Thread State	39
A.3	WCP Lock State	39
A.4	Alloc State	40
A.5	Block State	40
B	Code and Implementation	40

1 Introduction

1.1 Background and Motivation

Concurrent programs offer performance through parallelism, but introduce subtle correctness hazards. Chief among these is the **data race**: two threads accessing the same memory location concurrently, with at least one access being a write, and no synchronization ordering those accesses. Data races lead to non-deterministic, hard-to-reproduce bugs that standard testing rarely catches.

Dynamic race detectors instrument a program and monitor every memory access at runtime. When two accesses to the same location are not ordered by any ordering relation, a race is reported. The dominant algorithmic tool for this task is the **vector clock**.

1.2 The Problem with Traditional Vector-Clock Based Detection

A vector clock (VC) assigns each thread t a vector $C_t \in \mathbb{N}^k$ (where k is the number of threads), and every synchronization event updates this vector. A race on variable x is detected when the current information of the accessing thread t regarding other threads who might be accessing x is not up to date. Thread t does it by comparing its own knowledge (C_t) with the recent reader/writer if the variables x is trying to access.

While theoretically elegant, this approach carries two major practical costs:

1. **Vector clock comparisons are $O(k)$.** With k threads, every memory access requires a full k -dimensional comparison. In programs with many threads, this becomes the dominant overhead.
2. **Redundant checks.** In practice, most memory accesses are *thread-local*—the same thread accesses the same variable repeatedly with no other thread involved. Yet naive algorithms perform full VC comparisons on every such access.

Two types of redundancy appear in practice:

Redundant same-epoch access. Consider thread T_1 reading variable x multiple times in a row without any concurrent access. FastTrack’s epoch optimization avoids the full VC comparison if the variable is already stamped with the current epoch of T_1 . However, the check still runs on every access when current threads updates epoch on each access. Eg. when locking and unlocking x continuously.

Repeated mutex lock acquires. In case of lock based implementation of race detectors. Every call to the read or write callbacks involve acquiring two mutex locks one involving the thread metadata and another involving the variable metadata. This thing can be avoided if we have some lock free mechanism.

Figure 1 illustrates these redundancies with a simple interleaving.

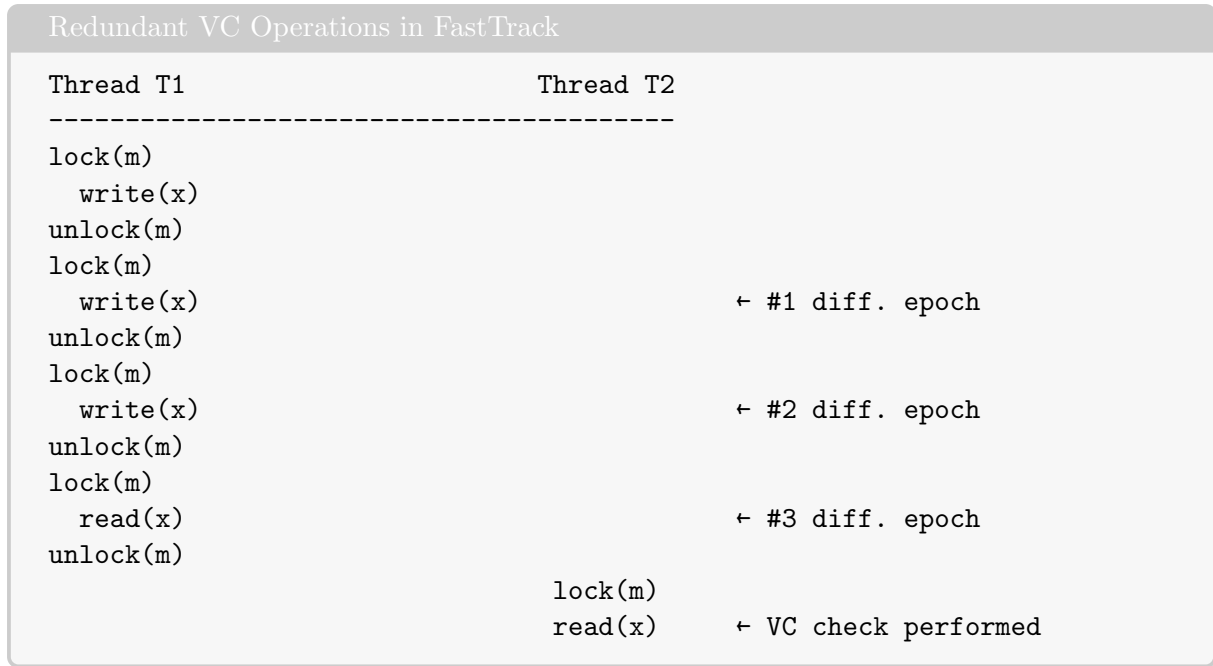


Figure 1: Redundant vector clock checks. Checks #1–#3 involve no cross-thread sharing, yet each calls the slow race detector callbacks. Sharing Analysis eliminates these.

1.3 Our Approach: Sharing Analysis

Sharing Analysis (SA) is a lightweight per-variable state machine that tracks whether a variable is currently *exclusively owned* by one thread or genuinely *shared* across multiple threads. The key idea: if a variable is exclusively owned by the accessing thread, there cannot be a race—so the expensive vector-clock machinery can be skipped entirely **even if the epoch is different**.

The state machine has four states per variable:

UNACCESSED → OWNED_READ / OWNED_WRITE → SHARED

State transitions are performed via atomic compare-and-swap (CAS) on a single 64-bit “hot word”, making the fast path **lock-free**. Only when the state is **SHARED**—or when a transition out of **OWNED** is detected—does the full VC machinery run.

This report studies SA applied to two algorithms:

- **FastTrack**: HB-based race detector with epoch optimization.
- **WCP (Weak Causally Precedes)**: a predictive race detector that can find races not detectable by HB by tracking a weaker causality relation.

2 Preliminaries

2.1 FastTrack: HB-Based Race Detection

2.1.1 The Happens-Before Relation

Definition 2.1 (Happens-Before). *Given an execution trace σ , the happens-before relation $\xrightarrow{HB}$ is the smallest partial order such that:*

1. If e and f are events in the same thread and e occurs before f , then $e \xrightarrow{HB} f$.
2. If $e = \text{rel}(\ell)$ and $f = \text{acq}(\ell)$ for the same lock ℓ and e precedes f , then $e \xrightarrow{HB} f$.
3. If $e = \text{fork}(t')$ and f is any event in thread t' , then $e \xrightarrow{HB} f$.
4. If e is any event in thread t' and $f = \text{join}(t')$, then $e \xrightarrow{HB} f$.
5. HB is transitively closed.

Two accesses to the same variable x form a **data race** if neither happens before the other and at least one is a write.

2.1.2 Vector Clocks and Epochs

Each thread t maintains a vector clock $C_t \in \mathbb{N}^k$. The key operations are:

$$\begin{aligned} (C \sqcup D)[i] &= \max(C[i], D[i]) \quad (\text{join}) \\ C \sqsubseteq D &\iff \forall i. C[i] \leq D[i] \quad (\text{ordering}) \end{aligned}$$

An **epoch** is a compact representation (t, c) encoding thread t at clock value c . The epoch of thread t is $\epsilon_t = (t, C_t[t])$. FastTrack exploits the observation that most variables are read or written by the same thread consecutively, so storing just the last writer's epoch suffices for the common case.

2.1.3 FastTrack Algorithm

Each variable x maintains:

- W_x : epoch of the last write.
- R_x : either a single read epoch (exclusive mode) or a special `READ_SHARED` sentinel triggering full vector clock storage Rvc_x .

FastTrack Read Rule

read(t, x):

1. **Same epoch check**: if $R_x = \epsilon_t$, return (no-op).
2. **W-R race check**: if $W_x.\text{clock} > C_t[W_x.\text{tid}]$, report race.
3. **Update R_x** : if the previous reader's epoch is $\sqsubseteq C_t$, set $R_x := \epsilon_t$ (exclusive); else transition to `READ_SHARED` mode.

FastTrack Write Rule

write(t, x):

1. **Same epoch check:** if $W_x = \epsilon_t$, return.
2. **W-W race check:** if $W_x.\text{clock} > C_t[W_x.\text{tid}]$, report race.
3. **R-W race check:** compare R_x (or Rvc_x in shared mode) against C_t .
4. **Update:** $W_x := \epsilon_t$; clear R_x .

Lock events:

- **acquire**(ℓ): $C_t := C_t \sqcup C_\ell$.
- **release**(ℓ): $C_\ell := C_t$; $C_t[t]++$.

2.2 WCP: Predictive Race Detection

2.2.1 Motivation for Predictive Detection

HB-based detectors are *sound* (no false positives) but *incomplete*: they only report races that *actually manifest* in the observed execution. A single change in thread interleaving—without altering the program—can expose a race that HB misses entirely. Figure 2 shows such a trace and its causal structure.

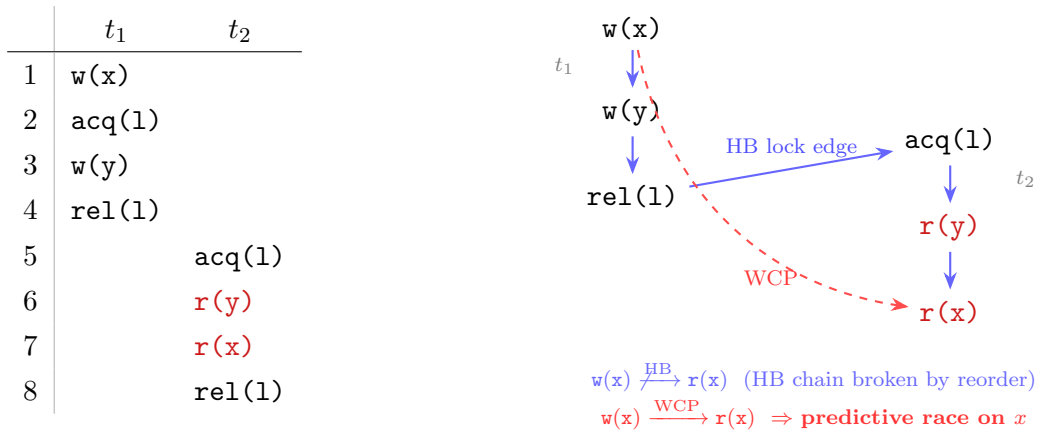


Figure 2: Standard Happens-Before (HB) misses the data race on x because the lock handoff ($\text{rel}(1) \rightarrow \text{acq}(1)$) creates an ordering. However, since $w(x)$ occurs outside the critical section, the lock does not protect it. The Weak Causally Precedes (WCP) model ignores this artificial lock edge, identifying that $w(x)$ and $r(x)$ could execute concurrently in a valid alternate schedule, successfully catching the predictive race.

2.2.2 The WCP Relation

Definition 2.2 (Weak Causally Precedes (WCP)). *relaxes traditional synchronization constraints. Unlike the strict happens-before relation, which enforces a rigid chronological sequence across all lock transfers, WCP only dictates an ordering between critical sections if they contain conflicting accesses to shared data. WCP allows inferring alternative thread interleavings from a single observed trace.*

Informally: WCP allows inferring ordering constraints through critical sections that share variables, beyond what pure lock-based HB can establish. This lets WCP detect **predictive data races**—races that would appear under a valid alternative scheduling.

2.2.3 WCP State

Each thread t maintains three clocks:

- N_t : a local integer clock, incremented after each release.
- H_t : HB vector clock (the usual FastTrack-style VC).
- P_t : WCP-predecessor vector clock.
- $C_t = P_t[t \mapsto N_t]$: the effective WCP clock used in race checks.

Each lock ℓ maintains:

- H_ℓ, P_ℓ : HB and WCP clocks of the last release.
- $Acq_\ell(t), Rel_\ell(t)$: FIFO queues of clock snapshots enabling WCP-predecessor propagation.
- $Lr[\ell][x], Lw[\ell][x]$: per-variable join of release HB clocks for reads/writes in the critical section.

2.2.4 WCP Procedures

WCP Acquire

acquire(t, ℓ):

1. $H_t := H_t \sqcup H_\ell$
2. $P_t := P_t \sqcup P_\ell$
3. $\forall t' \neq t: Acq_\ell(t').enqueue(C_t)$

WCP Release

release(t, ℓ, R, W):

1. While $Acq_\ell(t).front() \sqsubseteq C_t$: dequeue from $Acq_\ell(t)$ and $P_t := P_t \sqcup Rel_\ell(t).dequeue()$
2. $\forall x \in R: Lr[\ell][x] := Lr[\ell][x] \sqcup H_t$
3. $\forall x \in W: Lw[\ell][x] := Lw[\ell][x] \sqcup H_t$
4. $H_\ell := H_t; P_\ell := P_t$
5. $\forall t' \neq t: Rel_\ell(t').enqueue(H_t)$
6. N_t++

WCP Read/Write Rules

read(t, x, L) where $L = \text{held locks}$:

- $P_t := P_t \sqcup \bigsqcup_{\ell \in L} Lw[\ell][x]$
- Race if $\neg(W_x \sqsubseteq C_t)$ (W-R race)
- $R_x := R_x \sqcup C_t$

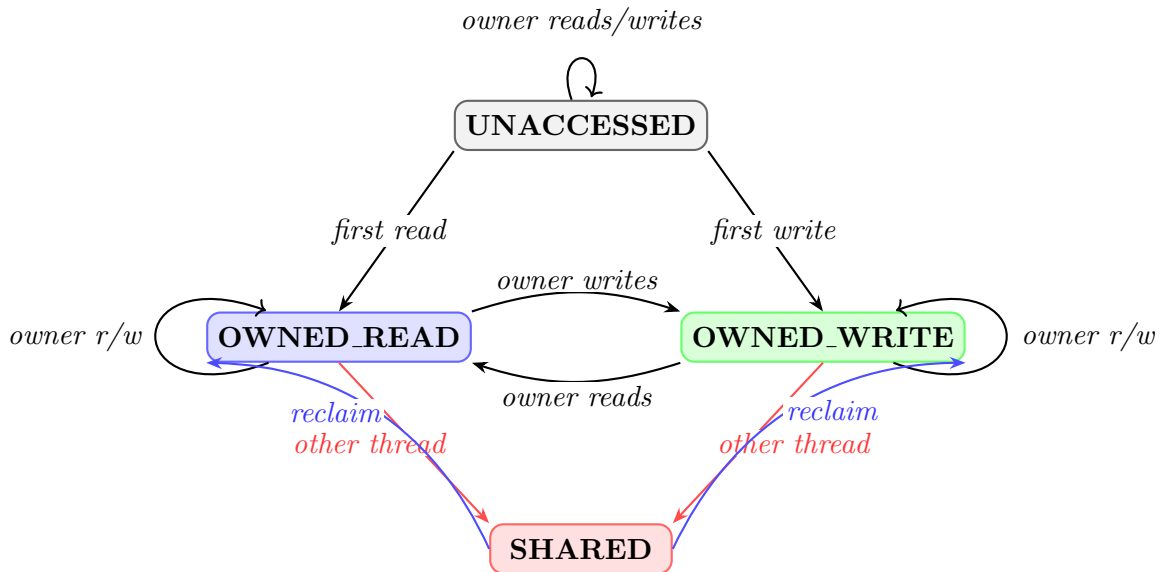
write(t, x, L):

- $P_t := P_t \sqcup \bigsqcup_{\ell \in L} (Lr[\ell][x] \sqcup Lw[\ell][x])$
- Race if $\neg(R_x \sqsubseteq C_t)$ (R-W) or $\neg(W_x \sqsubseteq C_t)$ (W-W)
- $W_x := W_x \sqcup C_t$

3 Sharing Analysis: The Optimization Layer

3.1 Core Idea

Sharing Analysis maintains, per memory location x , a state machine with four states:



The fast path check is: load the 64-bit *hot word* (atomically), inspect the state and owner field, and if the current thread owns the variable, skip the VC machinery entirely and return.

3.2 The Hot Word

The hot word encodes both the current owner TID and the state into a single 64-bit integer:

Listing 1: Hot word encoding (fasttrack_runtime_with_sa.cpp)

```

1 bits [63:32] = owner_tid (int32, -1 means no owner)
2 bits [31: 0] = ShareState (uint32: 0=UNACCESSED, 1=OWNED_READ,
3                               2=OWNED_WRITE, 3=SHARED)

```

3.3 Shadow Table

The shadow table is a flat array of `ShadowEntry` structures, each exactly one cache line (64 bytes) in size to prevent false sharing. `ShadowEntry` maps a variable to its metadata. Metadata information is extracted via `get_shadow_entry` method. Entries are located via Knuth's multiplicative hash:

Listing 2: Shadow table entry and lookup (`fasttrack_runtime_with_sa.cpp`)

```

1 struct alignas(64) ShadowEntry {
2     std::atomic<uint64_t> hot_word { pack_hot(-1, ShareState::
3         UNACCESSED) };
4     std::atomic<VarState*> state { nullptr };
5     std::atomic<uintptr_t> key { 0 };
6 };
7
8 #define SHADOW_SIZE (1 << 20) // 1M entries
9 #define SHADOW_MASK (SHADOW_SIZE - 1)
10
11 static ShadowEntry* get_shadow_entry(void* addr) {
12     uintptr_t key = ((uintptr_t)addr >> 2) + 1; // 4-byte
13     granularity
14     size_t slot = (key * 2654435761ULL) & SHADOW_MASK; //
15     Knuth hash
16     for (;;) {
17         uintptr_t cur = shadow_table[slot].key.load(std::
18             memory_order_acquire);
19         if (cur == key) return &shadow_table[slot];
20         if (cur == 0) {
21             uintptr_t expected = 0;
22             if (shadow_table[slot].key.compare_exchange_strong(
23                 expected, key, std::memory_order_acq_rel))
24                 return &shadow_table[slot];
25             continue;
26         }
27         slot = (slot + 1) & SHADOW_MASK; // linear probing on
28         collision
29     }
30 }

```

3.4 Sharing Analysis Applied to FastTrack

3.4.1 Owner Epoch Cache

When a variable is in `OWNED_*` state, the slow path (with full VC checks) is never called. But when the variable eventually transitions to `SHARED`, the slow path needs accurate W_x and R_x values. To reconstruct them, `VarState` caches the most recent owner epoch for both reads and writes:

Listing 3: VarState with owner epoch cache (fasttrack_runtime_with_sa.cpp)

```

1
2 struct VarState {
3     // --- Fasttrack specific metadata ---
4     // ...
5     // -----
6
7     std::atomic<Epoch> owner_write_epoch { 0 }; // last write
        epoch while OWNED
8     std::atomic<Epoch> owner_read_epoch  { 0 }; // last read
        epoch while OWNED
9 };

```

3.4.2 Fast Path for FastTrack+SA

The fast path for reads is:

Listing 4: FastTrack+SA fast-path read (__ft_read)

```

1 void __ft_read(void* addr, int line_no, char* var_name) {
2     ThreadState* t = get_current_thread();
3     ShadowEntry* e = get_shadow_entry(addr);
4
5     uint64_t hw = e->hot_word.load(std::memory_order_acquire);
6     ShareState ss = hot_state(hw);
7     int owner = hot_owner(hw);
8
9     // FAST PATH: current thread owns the variable
10    if ((ss == ShareState::OWNED_READ || ss == ShareState::
        OWNED_WRITE)
11        && owner == t->tid) {
12        VarState* x = e->state.load(std::memory_order_acquire);
13        if (x) {
14            x->owner_read_epoch.store(
15                t->self_epoch.load(std::memory_order_relaxed),
16                std::memory_order_relaxed);
17        }
18        uint64_t desired = hw;
19        // CAS validates ownership hasn't been stolen since the
        load
20        if (e->hot_word.compare_exchange_strong(hw, desired,
21            std::memory_order_release, std:::
        memory_order_relaxed))
22            return; // SUCCESS: no VC work at all
23    }
24    // SLOW PATH: fall through to full FT machinery
25    ft_slow_read(addr, line_no, e, t, var_name);
26 }

```

The case for __ft_write will be very similar.

3.4.3 Slow Path: OWNED-to-SHARED Transition

The most interesting case is when a second thread accesses a variable currently in `OWNED_WRITE` state. The slow path must restore W_x/R_x from the owner cache before running the FT race check. If there is no race then we will try to claim back ownership (for both read and write):

Listing 5: FT+SA slow path: ownership transition to SHARED

```

1 case ShareState::UNACCESSED:
2     // First time access: update metadata and return
3 case ShareState::OWNED_READ:
4 case ShareState::OWNED_WRITE:
5     // Different thread access: restore W/R from owner cache,
6     // transition to SHARED
7     x->W.store(x->owner_write_epoch.load(std::
8         memory_order_acquire), ...);
9     x->R.store(x->owner_read_epoch.load(std::memory_order_acquire
10         ), ...);
11     x->Rvc.clear();
12     e->hot_word.store(pack_hot(t->tid, ShareState::SHARED), ...);
13     {
14         // Fasttrack callback to check races
15         bool raced = ft_write_core(addr, line_no, x, t, var_name)
16         ;
17         // Try to reclaim ownership after the check
18         if (!raced && can_reclaim(t, x)) {
19             // Update metadata to get back ownership
20         }
21     }

```

3.4.4 Reclaim: Returning to OWNED State

After a `SHARED`-state check, if the current thread has “seen” all previous accesses (i.e., all prior writers and readers are causally before t), it can reclaim exclusive ownership:

Listing 6: `can_reclaim` check (`fasttrack_runtime_with_sa.cpp`)

```

1 static bool can_reclaim(ThreadState* t, VarState* x) {
2     Epoch W = x->W.load(std::memory_order_relaxed);
3     if (W != 0) {
4         int w_tid = get_tid(W), w_clock = get_clock(W);
5         // Another thread's write not yet seen -> cannot reclaim
6         if (w_tid != t->tid && w_clock > t->get_clock_of(w_tid))
7             return false;
8     }
9     Epoch R = x->R.load(std::memory_order_relaxed);
10    if (R == READ_SHARED) {
11        for (int i = 0; i < (int)x->Rvc.size(); ++i) {
12            if (x->Rvc[i] == 0) continue;
13            if (i != t->tid && t->get_clock_of(i) < get_clock(x->
14                Rvc[i]))
15                return false;
16        }
17    }
18    return true;
19 }

```

```

15     }
16   } else if (R != 0) {
17       int r_tid = get_tid(R), r_clock = get_clock(R);
18       if (r_tid != t->tid && t->get_clock_of(r_tid) < r_clock)
19           return false;
20   }
21   return true;
22 }

```

3.5 Sharing Analysis Applied to WCP

3.5.1 Key Differences from FT+SA

Because WCP uses full vector clocks (not scalar epochs) for W_x and R_x , the owner cache stores C_t snapshots rather than epochs:

Listing 7: WCP VarState with VClock owner cache (wcp_runtime_with_sa.cpp)

```

1 struct VarState {
2     VClock Wx;           // join of C_t for all writes (slow
3                          // path)
4     VClock Rx;           // join of C_t for all reads (slow
5                          // path)
6     // Owner cache: C_t snapshot from last time the owning thread
7     // wrote/read while in OWNED_* state. Replayed into Wx/Rx on
8     // transition to SHARED so no history is lost.
9     VClock owner_write_Ct;
10    VClock owner_read_Ct;
11
12    std::recursive_mutex mtx;
13 };

```

3.5.2 WCP Fast Path

The fast path is structurally identical to FT+SA. When the variable is OWNED by the current thread, it updates the owner VClock cache and returns without any WCP work:

Listing 8: WCP+SA fast-path read (__wcp_read)

```

1 void __wcp_read(void* addr, int line_no, char* var) {
2     ThreadState* t = get_current_thread();
3     ShadowEntry* e = get_shadow_entry(addr);
4
5     uint64_t hw = e->hot_word.load(std::memory_order_acquire);
6     ShareState ss = hot_state(hw);
7     int owner = hot_owner(hw);
8
9     if ((ss == ShareState::OWNED_READ || ss == ShareState::
10        OWNED_WRITE)
11        && owner == t->tid) {
12         VarState* x = e->state.load(std::memory_order_acquire);

```

```

12     if (x) {
13         std::lock_guard<std::recursive_mutex> lk(x->mtx);
14         x->owner_read_Ct = t->Ct(); // update VClock cache
15     }
16     // Still record in CS frames (WCP requires all accesses
17     // tracked for Lr/Lw)
18     csframe_record_read(t->tid, (uintptr_t)addr);
19
20     uint64_t desired = hw;
21     if (e->hot_word.compare_exchange_strong(hw, desired,
22         std::memory_order_release, std::memory_order_relaxed))
23         return;
24 }
25 wcp_sa_slow_read(addr, line_no, e, t, var);
26 }

```

3.5.3 WCP+SA Slow Path: Restoring VClock History

The transition from OWNED_WRITE to SHARED for WCP is more nuanced than FT+SA because the owner cache holds full VClocks:

Listing 9: WCP+SA: OWNED-to-SHARED transition in wcp_sa_slow_read

```

1 if (ss == ShareState::UNACCESSED)
2     // First time access: Update metadata and return
3 if (ss == ShareState::OWNED_WRITE) {
4     // Previous owner was writing: restore Wx from write cache;
5     Rx=empty
6     x->Wx = x->owner_write_Ct;
7     x->Rx.clear();
8 } else {
9     // Previous owner was reading: restore both caches
10    x->Wx = x->owner_write_Ct;
11    x->Rx = x->owner_read_Ct;
12 }
13 e->hot_word.store(pack_hot(t->tid, ShareState::SHARED), std::memory_order_release);
14 {
15     bool raced = wcp_read_core(addr, line_no, x, t, var);
16     if (!raced && can_reclaim(t, x)) {
17         // Update metadata to reclaim ownership
18     }
19 }

```

3.5.4 WCP+SA Reclaim

The WCP reclaim check uses VClock comparisons instead of epoch comparisons:

Listing 10: WCP+SA can_reclaim (wcp_runtime_with_sa.cpp)

```

1 static bool can_reclaim(ThreadState* t, VarState* x) {

```

```

2   VClock Ct = t->Ct();
3   // All previous writers must be causally before t under WCP
4   if (!(x->Wx <= Ct)) return false;
5   // All previous readers must be causally before t under WCP
6   if (!(x->Rx <= Ct)) return false;
7   return true;
8 }

```

Note that the WCP fast path additionally must call `csframe_record_read/write` even in the fast path. This is a key difference from FT+SA: WCP uses the CS frame tracker to populate $Lr[\ell][x]$ and $Lw[\ell][x]$ at release time, and these must track *all* accesses, not just slow-path accesses.

4 Dry Run Examples

We present two carefully designed trace examples. The first proves **correctness**: every algorithm detects the same set of races. The second proves **efficiency**: the sharing-analysis variants invoke the slow vector-clock handler far fewer times than the plain algorithms.

4.1 Example 1 – Correctness: Same Races Across All Four Algorithms

4.1.1 Execution trace

We fix the execution order: E1, E2, $\text{unlock}(m_1 \text{ by } T_1)$, E4, E5, $\text{unlock}(m_1 \text{ by } T_2)$, E6, E3. Initial vector clocks: $C_{T_1} = [1, 0, 0]$, $C_{T_2} = [0, 1, 0]$, $C_{T_3} = [0, 0, 1]$. All epochs are written as (T_i, c) meaning thread i at clock value c .

Evt	Operation	C_{T_1}	C_{T_2}	C_{T_3}	Variable / Lock State
E1	T_1 : write(x)	[1,0,0]	[0,1,0]	[0,0,1]	$W_x \leftarrow (T_1, 1)$; $R_x = \emptyset$
E2	T_1 : lock(m_1)	[1,0,0]	[0,1,0]	[0,0,1]	$C_{T_1} \sqcup L_{m_1} = [0, 0, 0]$ – no change
	T_1 : write(y)	[1,0,0]	[0,1,0]	[0,0,1]	$W_y \leftarrow (T_1, 1)$; $R_y = \emptyset$
	T_1 : unlock(m_1)	[2,0,0]	[0,1,0]	[0,0,1]	$L_{m_1} \leftarrow [1, 0, 0]$; $C_{T_1}[1] += 1$
E4	T_2 : read(x)	[2,0,0]	[0,1,0]	[0,0,1]	$W_x = (T_1, 1)$; $C_{T_2}[T_1] = 0 < 1$ Race W-R on x: $T_1 \leftrightarrow T_2$
		[2,0,0]	[0,1,0]	[0,0,1]	$R_x \leftarrow (T_2, 1)$ (exclusive read)
E5	T_2 : lock(m_1)	[2,0,0]	[1,1,0]	[0,0,1]	$C_{T_2} \sqcup L_{m_1} = [1, 0, 0]$; $C_{T_2}[T_1] = 1$
	T_2 : write(y)	[2,0,0]	[1,1,0]	[0,0,1]	$W_y = (T_1, 1)$; $C_{T_2}[T_1] = 1 \geq 1$ – no W-W $R_y = \emptyset$ – no R-W (no race: T_2 has seen T_1 's unlock)
	T_2 : unlock(m_1)	[2,0,0]	[1,2,0]	[0,0,1]	$L_{m_1} \leftarrow [1, 1, 0]$; $C_{T_2}[2] += 1$
E6	T_3 : write(z)	[2,0,0]	[1,2,0]	[0,0,1]	$W_z \leftarrow (T_3, 1)$; $R_z = \emptyset$
E3	T_1 : read(z)	[2,0,0]	[1,2,0]	[0,0,1]	$W_z = (T_3, 1)$; $C_{T_1}[T_3] = 0 < 1$ Race W-R on z: $T_3 \leftrightarrow T_1$

Table 1: Correctness trace: event-by-event vector clock evolution

Detected races in this trace.

1. **W-R on x** (E1 vs E4): T_1 writes x at clock 1; T_2 reads x without having seen any synchronization with T_1 . $C_{T_2}[T_1] = 0 < 1$.
2. **W-R on z** (E6 vs E3): T_3 writes z at clock 1; T_1 reads z without synchronizing with T_3 . $C_{T_1}[T_3] = 0 < 1$.

4.1.2 Algorithm-by-Algorithm Trace for Correctness Example

Plain FastTrack. Every memory access enters the slow handler unconditionally (no same-epoch shortcuts fire in this trace because no thread repeats an access to an already-stamped variable at the same epoch):

- E1 write(x): slow write. $W_x \leftarrow (T_1, 1)$. No prior write/read. **No race.**
- E2 write(y): slow write inside CS. $W_y \leftarrow (T_1, 1)$. **No race.**
- E4 read(x): slow read. $W_x = (T_1, 1)$, $C_{T_2}[T_1] = 0 < 1$. **W-R race on x .** $R_x \leftarrow (T_2, 1)$.
- E5 write(y): slow write inside CS. After lock join: $C_{T_2}[T_1] = 1 \geq 1$. $W_y = (T_1, 1)$ – no W-W. $R_y = \emptyset$ – no R-W. $W_y \leftarrow (T_2, 2)$. **No race.**
- E6 write(z): slow write. $W_z \leftarrow (T_3, 1)$. **No race.**
- E3 read(z): slow read. $W_z = (T_3, 1)$, $C_{T_1}[T_3] = 0 < 1$. **W-R race on z .**

FastTrack: 2 races detected. Slow calls: 4 writes + 2 reads = 6.

Plain WCP. WCP additionally computes $C_t = P_t[t \mapsto N_t]$ and propagates P_t through $Lw[\ell][x]$ at every read/write. The extra bookkeeping reveals:

- At release of m_1 by T_1 : $Lw[m_1][y] \leftarrow H_{T_1} = [1, 0, 0]$.
- At lock of m_1 by T_2 (E5): $P_{T_2} \sqcup Lw[m_1][y] = [1, 0, 0]$, so $C_{T_2} = P_{T_2}[T_2 \mapsto N_{T_2}]$.
- At write(y) by T_2 : WCP checks $\neg(W_y^{WCP} \sqsubseteq C_{T_2})$. Because W_y^{WCP} was set from T_1 's C_{T_1} which included $N_{T_1} = 1$ in position T_1 , and P_{T_2} now carries this, the WCP predecessor relation orders this write. However, the *acquire* of m_1 by T_2 happens *after* T_1 's release, so both HB and WCP agree here.
- Same W-R races on x and z as FastTrack.

WCP: 2 HB races detected (same as FT). Slow calls: 6 (all events enter slow path).

FastTrack + Sharing Analysis. SA tracks per-variable ownership. In this trace, every variable is accessed by exactly two distinct threads and with no repeated same-thread accesses before the cross-thread event:

- E1 write(x) by T_1 : x is UNACCESSED. Slow path. $x \rightarrow \text{OWNED_WRITE}(T_1)$. **1 slow write.**
- E2 write(y) by T_1 (inside CS): y is UNACCESSED. Slow path. $y \rightarrow \text{OWNED_WRITE}(T_1)$. **1 slow write.**
- E4 read(x) by T_2 : x is $\text{OWNED_WRITE}(T_1)$, owner $\neq T_2$. Slow path. Restore epoch cache: $W_x \leftarrow \text{owner_write_epoch} = (T_1, 1)$. FT check: $C_{T_2}[T_1] = 0 < 1$. **W-R race on x .** Reclaim? $W_x = (T_1, 1)$ not seen by T_2 (race was just found) $\rightarrow$ stay SHARED. **1 slow read.**
- E5 write(y) by T_2 (inside CS, after lock join): y is $\text{OWNED_WRITE}(T_1)$, owner $\neq T_2$. Slow path. Restore: $W_y = (T_1, 1)$. After lock join $C_{T_2}[T_1] = 1 \geq 1$ – no W-W. Reclaim check: T_2 has seen W_y – reclaim succeeds. $y \rightarrow \text{OWNED_WRITE}(T_2)$. **1 slow write.**
- E6 write(z) by T_3 : z is UNACCESSED. Slow path. $z \rightarrow \text{OWNED_WRITE}(T_3)$. **1 slow write.**
- E3 read(z) by T_1 : z is $\text{OWNED_WRITE}(T_3)$, owner $\neq T_1$. Slow path. Restore epoch cache: $W_z = (T_3, 1)$. $C_{T_1}[T_3] = 0 < 1$. **W-R race on z .** **1 slow read.**

FT+SA: 2 races detected. Slow calls: 4 writes + 2 reads = 6. The race count is identical to plain FastTrack.

WCP + Sharing Analysis. Identical state machine to FT+SA; owner cache stores VClock snapshots instead of epochs. Every event in this trace hits the slow path for the same reason as FT+SA (each variable’s first access from a new thread forces the transition). CS frames are updated even on the fast path (though the fast path never fires here). Race count and slow call count are identical to WCP.

WCP+SA: 2 races detected. Slow calls: 6.

4.1.3 Correctness Summary

Algorithm	W-R on x	W-R on z	Total HB Races	Total Slow Calls
FastTrack	✓	✓	2	6
WCP	✓	✓	2	6
FastTrack + SA	✓	✓	2	6
WCP + SA	✓	✓	2	6

Table 2: Correctness example: all four algorithms detect the same 2 HB races with the same number of slow-path invocations. This confirms SA does not suppress any race.

Key insight: Sharing Analysis is a pure optimisation layer. It changes *when* the slow-path machinery runs, not *what* it computes. Whenever a new thread accesses a variable that is currently OWNED by another thread, the slow path fires and the owner-epoch cache is replayed into W_x/R_x (or W_x^{vc}/R_x^{vc} for WCP) before the race check. The race check therefore sees exactly the same state it would have seen had every access gone through the slow path, guaranteeing the same set of races is reported.

4.2 Example 2 – Efficiency: Repeated Lock/Unlock Exposing the SA Advantage

4.2.1 Interleaving and Vector Clock Trace

Execution order: all of T_1 's loop body first, then all of T_2 's loop body, then T_1 's final unprotected write. Initial clocks: $C_{T_1} = [1, 0]$, $C_{T_2} = [0, 1]$. Epochs are written (T_i, c) . L_m denotes the lock's stored vector clock after the most recent release.

Evt	Operation	C_{T_1}	C_{T_2}	State after event
— T_1 iteration 1 —				
A1	T_1 : lock(m)	[1,0]	[0,1]	join $L_m = [0, 0]$: no change
A2	T_1 : write(x)	[1,0]	[0,1]	$W_x \leftarrow (T_1, 1)$; $\epsilon_{T_1} = (T_1, 1)$
A3	T_1 : unlock(m)	[2,0]	[0,1]	$L_m \leftarrow [1, 0]$; $C_{T_1}[1]++$ $\epsilon_{T_1} = (T_1, 2)$
— T_1 iteration 2 —				
B1	T_1 : lock(m)	[2,0]	[0,1]	join $L_m = [1, 0]$: $C_{T_1}[1]$ already ≥ 1 , no change
B2	T_1 : write(x)	[2,0]	[0,1]	$W_x = (T_1, 1) \neq \epsilon_{T_1} = (T_1, 2)$: FT same-epoch miss $C_{T_1}[T_1] = 2 \geq 1$ (saw own write): no W-W race $W_x \leftarrow (T_1, 2)$
B3	T_1 : unlock(m)	[3,0]	[0,1]	$L_m \leftarrow [2, 0]$; $\epsilon_{T_1} = (T_1, 3)$
— T_1 iteration 3 —				
C1	T_1 : lock(m)	[3,0]	[0,1]	join $L_m = [2, 0]$: no change
C2	T_1 : write(x)	[3,0]	[0,1]	$W_x = (T_1, 2) \neq \epsilon_{T_1} = (T_1, 3)$: FT same-epoch miss $C_{T_1}[T_1] = 3 \geq 2$: no W-W race $W_x \leftarrow (T_1, 3)$
C3	T_1 : unlock(m)	[4,0]	[0,1]	$L_m \leftarrow [3, 0]$; $\epsilon_{T_1} = (T_1, 4)$
— T_2 iteration 1 (joins T_1 's full history) —				
D1	T_2 : lock(m)	[4,0]	[3,1]	join $L_m = [3, 0]$: $C_{T_2}[T_1] \leftarrow 3$; $\epsilon_{T_2} = (T_2, 1)$
D2	T_2 : write(x)	[4,0]	[3,1]	$W_x = (T_1, 3)$; $C_{T_2}[T_1] = 3 \geq 3$: no W-W $W_x \leftarrow (T_2, 1)$
D3	T_2 : unlock(m)	[4,0]	[3,2]	$L_m \leftarrow [3, 1]$; $\epsilon_{T_2} = (T_2, 2)$

Evt	Operation	C_{T_1}	C_{T_2}	State after event
— T_2 iteration 2 —				
E1	T_2 : lock(m)	[4,0]	[3,2]	join $L_m = [3, 1]$: $C_{T_2}[2]$ already ≥ 1 , no change
E2	T_2 : write(x)	[4,0]	[3,2]	$W_x = (T_2, 1) \neq \epsilon_{T_2} = (T_2, 2)$: FT same-epoch miss $C_{T_2}[T_2] = 2 \geq 1$: no W-W race $W_x \leftarrow (T_2, 2)$
E3	T_2 : unlock(m)	[4,0]	[3,3]	$L_m \leftarrow [3, 2]$; $\epsilon_{T_2} = (T_2, 3)$
— T_2 iteration 3 —				
F1	T_2 : lock(m)	[4,0]	[3,3]	join $L_m = [3, 2]$: no change
F2	T_2 : write(x)	[4,0]	[3,3]	$W_x = (T_2, 2) \neq \epsilon_{T_2} = (T_2, 3)$: FT same-epoch miss $C_{T_2}[T_2] = 3 \geq 2$: no W-W race $W_x \leftarrow (T_2, 3)$
F3	T_2 : unlock(m)	[4,0]	[3,4]	$L_m \leftarrow [3, 3]$; $\epsilon_{T_2} = (T_2, 4)$
— T_1 final unprotected write —				
G1	T_1 : write(x)	[4,0]	[3,4]	$W_x = (T_2, 3)$; $C_{T_1}[T_2] = 0 < 3$ W-W race on x: $T_2 \leftrightarrow T_1$

Table 3: Full vector clock trace. **FT same-epoch misses** are annotated at B2, C2, E2, F2 — these are the writes where T_1 (or T_2) re-enters the loop after an unlock has bumped their epoch, yet x has not been touched by any other thread.

4.2.2 Why FT’s Same-Epoch Check Fails Here

After each `unlock`, FastTrack increments the thread’s clock and recomputes $\epsilon_t = \text{make_epoch}(t, C_t[t])$. So even though T_1 is the only writer of x , the stored W_x epoch falls behind.

SA instead stores the *owner identity* in the hot word. Since T_2 never accesses x during T_1 ’s three iterations, the hot word reads `OWNED.WRITE( $T_1$ )` at B2 and C2, and the ownership check passes regardless of the epoch. The slow handler is never called.

4.2.3 Slow-Handler Count: All Four Algorithms

We count every event that requires a real vector-clock comparison (i.e., the slow handler is invoked). Lock/unlock events are not memory accesses and are excluded from the write/read count.

Plain FastTrack. FastTrack’s only fast path is the same-epoch check. As shown above, the epoch advances on every unlock, so the same-epoch check fails on every write to x in iterations 2 and 3 for both threads.

- **A2** write(x): $W_x = 0 \neq \epsilon_{T_1} \rightarrow$ **slow**. First write, sets $W_x = (T_1, 1)$.
- **B2** write(x): $W_x = (T_1, 1) \neq \epsilon_{T_1} = (T_1, 2) \rightarrow$ **slow**. No W-W (self-write). Sets $W_x = (T_1, 2)$.

- **C2** write(x): $W_x = (T_1, 2) \neq \epsilon_{T_1} = (T_1, 3) \rightarrow$ **slow**. Sets $W_x = (T_1, 3)$.
- **D2** write(x): $W_x = (T_1, 3) \neq \epsilon_{T_2} = (T_2, 1) \rightarrow$ **slow**. No W-W ($C_{T_2}[T_1] = 3 \geq 3$). Sets $W_x = (T_2, 1)$.
- **E2** write(x): $W_x = (T_2, 1) \neq \epsilon_{T_2} = (T_2, 2) \rightarrow$ **slow**. No W-W (self). Sets $W_x = (T_2, 2)$.
- **F2** write(x): $W_x = (T_2, 2) \neq \epsilon_{T_2} = (T_2, 3) \rightarrow$ **slow**. No W-W (self). Sets $W_x = (T_2, 3)$.
- **G1** write(x): $W_x = (T_2, 3) \neq \epsilon_{T_1} = (T_1, 4) \rightarrow$ **slow**. $C_{T_1}[T_2] = 0 < 3$: **W-W race**.

Plain FT: 7 slow writes, 0 slow reads, 7 total VC checks. 0 fast-path hits.

Plain WCP. WCP has no same-epoch shortcut. Every memory access unconditionally enters the slow handler to join P_i from all held locks' Lw/Lr tables, and all 7 write events enter the slow path. Additionally, the per-lock CS tracking records the write to x inside each critical section, updating $Lw[m][x]$ on every release.

Plain WCP: 7 slow writes, 0 slow reads, 7 total VC checks. 0 fast-path hits.

In this trace FT and WCP have the same slow-call count because there are no repeated accesses that FT's same-epoch check would catch. Both algorithms call the slow handler on every write.

FastTrack + Sharing Analysis. SA checks ownership in the hot word before entering the slow path.

- **A2** write(x) by T_1 : hot word = UNACCESSED $\rightarrow$ **slow**. Transition to OWNED_WRITE(T_1). Sets $owner_write_epoch = (T_1, 1)$. **1 slow write**.
- **B2** write(x) by T_1 : hot word = OWNED_WRITE(T_1), owner = $T_1 \rightarrow$ **fast path**. Updates $owner_write_epoch = (T_1, 2)$. **0 VC ops**.
- **C2** write(x) by T_1 : hot word = OWNED_WRITE(T_1), owner = $T_1 \rightarrow$ **fast path**. Updates $owner_write_epoch = (T_1, 3)$. **0 VC ops**.
- **D2** write(x) by T_2 : hot word = OWNED_WRITE(T_1), owner $\neq T_2 \rightarrow$ **slow**. Restore $W_x \leftarrow owner_write_epoch = (T_1, 3)$. VC check: $C_{T_2}[T_1] = 3 \geq 3$: no W-W. Reclaim: T_2 has seen all of T_1 's writes $\rightarrow$ OWNED_WRITE(T_2). **1 slow write**.
- **E2** write(x) by T_2 : hot word = OWNED_WRITE(T_2), owner = $T_2 \rightarrow$ **fast path**. Updates $owner_write_epoch = (T_2, 2)$. **0 VC ops**.
- **F2** write(x) by T_2 : hot word = OWNED_WRITE(T_2), owner = $T_2 \rightarrow$ **fast path**. Updates $owner_write_epoch = (T_2, 3)$. **0 VC ops**.
- **G1** write(x) by T_1 : hot word = OWNED_WRITE(T_2), owner $\neq T_1 \rightarrow$ **slow**. Restore $W_x \leftarrow (T_2, 3)$. $C_{T_1}[T_2] = 0 < 3$: **W-W race**. **1 slow write**.

FT+SA: 3 slow writes, 0 slow reads, 3 total VC checks. 4 fast-path hits.
SA eliminates 4 out of 7 slow calls compared to plain FT — a **57% reduction**.

WCP + Sharing Analysis. WCP+SA uses the identical ownership hot-word check. The fast path still records accesses in the CS frame table (a hash-set insert, not a VC operation), but skips all P_t joins. The slow path is entered at exactly the same three events as FT+SA: A2, D2, G1.

WCP+SA: 3 slow writes, 0 slow reads, 3 total VC checks. 4 fast-path hits. Plain WCP needed 7. WCP+SA reduces this by **57%**.

4.2.4 Efficiency Summary

Algorithm	Slow Path	Fast Path	Races Found
FastTrack	7	0	1
WCP	7	0	1
FastTrack + SA	3	4	1
WCP + SA	3	4	1

Table 4: Slow VC-handler invocations for the lock-loop trace (7 write events). All four algorithms detect the same 1 race (W-W on x at G1). FT+SA and WCP+SA each eliminate 4 slow calls — a 57% reduction — by recognising that x is exclusively owned by the writing thread across iterations 2 and 3 of each loop.

The reduction scales with loop iteration count. For n loop iterations per thread the slow-call counts generalise to:

Algorithm	Slow calls (general n)	Fast-path hits
FT / WCP	$2n + 1$	0
FT+SA / WCP+SA	3	$2n - 2$

As n grows, the SA variants stay at a *constant* 3 slow calls while the plain variants grow linearly. For $n = 10$ iterations, plain FT/WCP require 21 slow calls; SA variants still require only 3.

5 Algorithmic Summary and Comparison

5.1 Structural Modifications for Sharing Analysis

5.1.1 FastTrack

The algorithmic changes to add SA to FastTrack are:

1. Add a `ShadowEntry` per variable with the hot word and state pointer.
2. Add `owner_write_epoch` and `owner_read_epoch` fields to `VarState`.
3. Wrap `__ft_read` and `__ft_write` with a hot-word check: if owned by current thread, update owner cache + CAS + return.

4. Move core FT logic to `ft_read_core/ft_write_core` (called only from slow path).
5. Add `can_reclaim` check after each slow-path call.

The lock-free fast path uses CAS to validate that ownership has not been stolen between the hot-word load and the update. This is important for correctness under concurrent ownership changes.

5.1.2 WCP

WCP+SA has all the above changes plus:

1. Owner cache uses `VClock` rather than `Epoch`: `owner_write_Ct` and `owner_read_Ct` store full vector clock snapshots.
2. `can_reclaim` uses `vc_leq` (full `VClock` comparison) instead of scalar epoch comparison.
3. The fast path *must* call `csframe_record_read/write` even when taking the fast path, since WCP requires all accesses to be tracked in CS frames for Lr/Lw updates at release. This is the key distinction: FT+SA's fast path does no VC work at all, but WCP+SA's fast path still performs the CS frame insertion (a simple hash-set insert).

6 Static Analysis : Motivation

A runtime approach while flexible and generalizes to all types of programs, still have the overhead of instrumented calls being made to the runtime library. A alternate approach which is less flexible and requires a strict and a much more formal treatment is static analysis, which can be performed using a LLVM pass. The goal is a hybrid approach. The static approach as discussed can not be precise, but even a very conservative approach here can provide real performance boost with 0 runtime overhead. But during testing, it was found that with small programs which are compiled using the `-O1` flag, the compiler already optimizes away all loads and store (and it is very good at that), so there are already few loads/stores to begin with, in this case the marginal utility of the pass decreases. Compare this with compilation with `-O0` flag, the pass reduces the instrumented calls by 70-80%. The next few sections highlight the problem, existing solutions, their limitations and the approach used in this project which is based on real compiler systems. [1]

7 The problem and approach

The Problem

Given program P and memory region R :
 Do **two different threads** ever both access R ?
 If yes $\Rightarrow$ **instrument this memory**.
 Determine this property statically.

This problem is undecidable, a very simple proof involves showing equivalence with **the halting problem** which is popularly proven to be undecidable.

- t_1 accesses $x \Leftrightarrow P_2$ halts
- t_1, t_2 both access x **iff** P_2 halts

```

1 int x;
2
3 void* t1(void*) {
4     // imaginary predicate:
5     if (halts(P2)) {
6         x = 1; // shared?
7     }
8 }
9
10 void* t2(void*) {
11     x = 2; // always runs
12 }

```

Not suprisingly, this is not merely a theoretical curiosity: many compiler optimisations share this undecidability boundary. Alias analysis, liveness, and points-to analysis are all undecidable in the general case.

7.1 The Approximation Strategy

The practical consequence is that we must choose an approximation. Two directions exist:

Approximation	Safety	Cost
Over-approximation (sound)	No missed races	Extra instrumentation (ok)
Under-approximation	<i>Missed races!</i>	Less instrumentation

Soundness is non-negotiable — missing a data race is a correctness bug, while adding redundant instrumentation is merely a performance issue. Therefore:

Design Goal

Compute a sound, provably correct over-approximation of the set of thread-local allocations that can be *safely skipped* during dynamic instrumentation. “Sound” means: if the analysis reports an allocation as local, it truly is local. False alarms (marking a local allocation as shared) are permitted; false negatives (marking a shared allocation as local) are not.

8 Escape Analysis — Background

8.1 What Is Escape Analysis?

Escape analysis is a pointer analysis technique that determines the *dynamic scope* of objects and pointers. For a given allocation site, it asks: *can this object be accessed outside the current function or thread?*

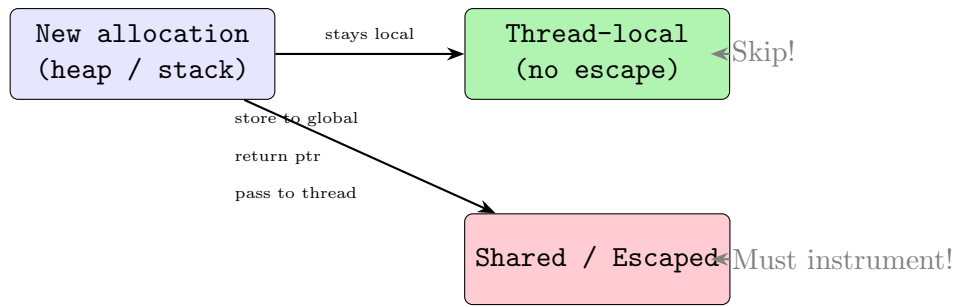


Figure 3: Decision flow for an allocation under escape analysis.

8.2 Classic Optimisations Enabled by Escape Analysis

Escape analysis unlocks a range of compiler optimisations:

1. **Stack allocation** — If a heap-allocated object does not escape the function, it can be promoted to a stack allocation, eliminating GC pressure or `malloc/free` overhead.
2. **Scalar replacement** — If the fields of an object never escape, each field can be replaced by an SSA value, eliminating the object entirely.
3. **Lock elision** — Synchronisation on a thread-local object is unnecessary; the lock can be removed.
4. **Atomic downgrade** — Memory operations on thread-local data can use relaxed ordering instead of sequentially consistent ordering.
5. **Instrumentation elision (our goal)** — Dynamic race detector instrumentation can be skipped for thread-local allocations.

8.3 Why Track Pointers Rather Than Values?

In C/C++, the only mechanism to share memory between threads is via pointers: a thread can only observe another thread’s data if it has a pointer to that data. This gives us a crucial simplification:

Key Observation

If the pointer value cannot be observed outside the current thread, then the contents of the pointed-to object cannot be observed outside the current thread either.

Therefore, tracking pointer escape is sufficient for sharing analysis.

This justifies an *intra-procedural* analysis: we track each pointer from its allocation site and monitor whether the pointer bits could be inspected by another thread.

9 Existing Solutions and Their Limitations

9.1 LLVM CaptureTracking API

LLVM already provides an API for pointer escape analysis:

Listing 11: LLVM CaptureTracking API signature

```
1 bool llvm::PointerMaybeCaptured(  
2     const Value *V,  
3     bool ReturnCaptures,  
4     bool StoreCaptures,  
5     unsigned MaxUsesToExplore  
6 );
```

What it answers. “Can bits of this pointer be observed outside the current *function* scope?” This is the `nocapture` property used in LLVM’s function attribute system.

How it works. The API walks all uses of the pointer:

- If the pointer is stored into any memory location $\Rightarrow$ **capture**.
- If the pointer is passed as a **nocapture** argument $\Rightarrow$ **no capture**.
- Derived pointers (`getelementptr`, `bitcast`, `addrspacecast`) produce aliases that are also tracked.

TSan itself uses this API to skip instrumentation of stack allocations (`alloca`) that the analysis proves are not captured.

9.2 Why CaptureTracking Is Insufficient

Despite being useful, CaptureTracking has a critical limitation for our purpose:

Limitation

CaptureTracking is **extremely conservative about stores**. Any store of the pointer — even into an object that is itself thread-local — is treated as a capture.

Consider the following pattern, which is common in practice:

Listing 12: Pattern that CaptureTracking over-approximates

```
1 struct Node { int val; };  
2 struct Container { Node* ptr; };  
  
3  
4 void foo() {  
5     Node* n = new Node();           // local allocation  
6     Container* c = new Container(); // also local  
7     c->ptr = n;                     // CaptureTracking: n is CAPTURED (  
        stored)  
8                                     // Reality: c is local, so n is  
                                    still local!  
9     n->val = 42;                    // CaptureTracking: must instrument  
10                                   // Our analysis: safe to skip  
11 }
```

Because CaptureTracking does not model the escape state of the container c , it cannot determine that storing n into c is safe. **The analysis solves this by maintaining a *points-to graph* that tracks which allocations live inside which other allocations, and propagates escape lazily only when the container itself escapes.**

10 Solution: ThreadEscapeAnalysis

10.1 Overview

implemented **ThreadEscapeAnalysis**: a flow-sensitive, intra-procedural LLVM pass based on abstract interpretation.

Core Design Principles

1. Maintain a set of *unescaped allocations* per basic block.
2. Track *aliases* (derived pointers) for each allocation.
3. Track a *points-to graph* — which allocations are stored inside which other allocations.
4. Propagate escape **lazily**: storing a local pointer into another local object does not immediately trigger escape.
5. Use a *worklist-based fixed-point* algorithm over the CFG for correctness in the presence of loops and branches.

10.2 Abstract State Representation

Each basic block B carries an **out-state** σ_B consisting of:

$$\sigma_B = \langle \mathcal{A}, \mathcal{P}, \mathcal{E} \rangle \quad (1)$$

where:

- $\mathcal{A} \subseteq \text{Allocs}$ — the set of *unescaped* allocations reachable at the end of B .
- $\mathcal{P} : \text{Alias} \rightarrow \text{Allocs} \times \mathbb{Z}$ — the *alias map*, mapping derived pointer SSA values to the base allocation and byte offset they point into.
- $\mathcal{E} \subseteq \text{Allocs}$ — the set of allocations confirmed escaped in B .

In addition, each allocation in $\mathcal{A}$ carries a *points-to map*:

$$PT(\alpha) : \text{Offset} \rightarrow \mathcal{A} \quad (2)$$

recording which other tracked allocations are stored into the fields of α .

10.2.1 State Transitions — Per Instruction

Allocation. When a new allocation is encountered (`alloca`, `malloc/new`, or `global`):

Listing 13: State trace: allocation

```
; State: A = {}
%a = call i8* @malloc(i64 16)
; State: A = {%a}
%b = alloca i32
; State: A = {%a, %b}
```

Globals are **never** added to $\mathcal{A}$; they are escaped by definition.

Alias derivation. Derived pointers produced by `getelementptr`, `bitcast`, `addrspacecast`, and `phi` nodes over tracked pointers are recorded in $\mathcal{P}$:

Listing 14: State trace: alias tracking

```
%a.8 = getelementptr i8, i8* %a, i32 8
; P: {%a.8 -> (%a, +8)}
%a.i32 = bitcast i8* %a.8 to i32*
; P: {%a.8 -> (%a, +8), %a.i32 -> (%a, +8)}
```

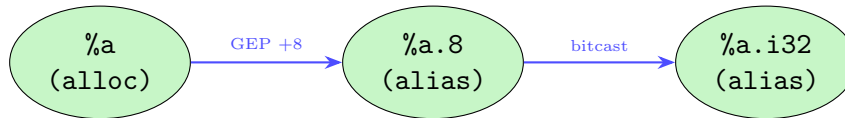


Figure 4: Alias chain from a root allocation. An escape of any alias propagates back to the root.

Store to unescaped object. Storing a tracked allocation β into a field of another tracked allocation α does *not* trigger escape. Instead, it records $\alpha.\text{field} \rightarrow \beta$ in the points-to graph:

Listing 15: State trace: points-to graph

```
store i8* %b, i8** %a.field_ptr
; PT(%a)[field_offset] = %b
; A = {%a, %b} -- both still local!
```

Escape events. An allocation α escapes when:

1. A pointer derived from α is **stored into a global** or a non-tracked memory location.
2. A pointer derived from α is **returned** from the function.
3. A pointer derived from α is **passed as an argument** to a function that may capture it (i.e. the parameter does not have `nocapture`).
4. All pointer arguments to `pthread_create` or any library functions without a definition.
5. A cast of pointer to `int` (since we can't track integer Value)

On escape of α , a cascade occurs: all allocations stored within α also escape, recursively.

Listing 16: State trace: escape cascade

```
; Before:  A = {%a, %b},  PT(%a)[0] = %b
store i8* %a, i8** @global_ptr    ; %a escapes (store to global)
; %a removed from A
; %b is in PT(%a), so %b also escapes
; After:   A = {},  E = {%a, %b}
```

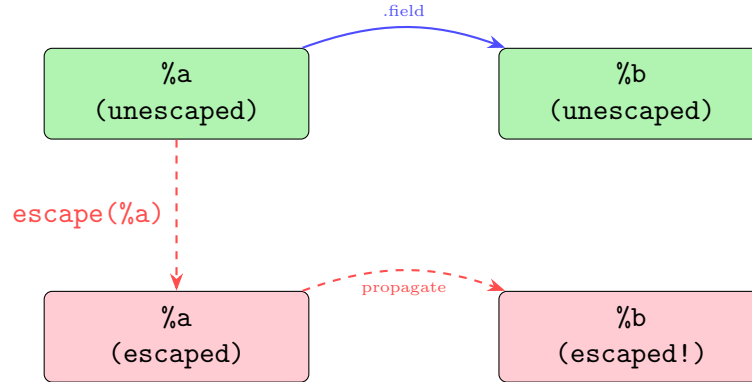


Figure 5: Escape cascade through the points-to graph. When $\%a$ escapes, $\%b$ — stored inside $\%a$ — escapes as a consequence.

A note on correctness

One may wonder, does the escape events list used here cover all the cases ? How is correctness never broken ?

No, the above list do not cover all the cases of escape. We are not LLVM experts and surely there might be cases where an escape case hasn't been handled. Now, as mentioned that the pass iterate over all the instructions in a BB, and handles each type of instruction separately, and creates points-to graph, Alias and create escape according to the type of instruction and its operands. The elegant part is for instructions that are not handled explicitly, and they have a tracked pointer operand, we assume the pointer escapes, acting like a final fallback thus never breaking soundness. Why this simple design is elegant one might ask, the reason is if in future we learn more about LLVM IR and discover a new escape event, handling it is as easy as adding another if condition and the precision of the final set can be increased very easily by just handling more types of instructions. It's worth mentioning that current cases cover most of the common cases, and for a real precision increase a inter-procedural analysis is required.

In short the missed cases only decrease precision and do not affect correctness

10.3 The Control-Flow Graph and Basic Block States

The analysis operates over LLVM's *Control-Flow Graph* (CFG), where nodes are basic blocks and edges represent possible control transfers. Each basic block has an associated state σ_B and the analysis propagates information forward along CFG edges.

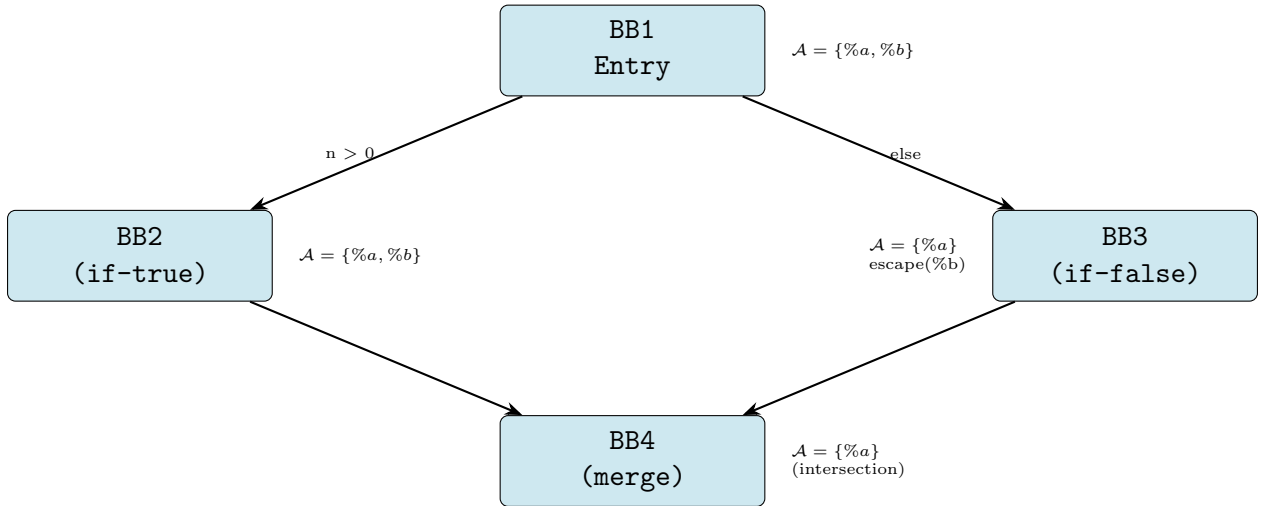


Figure 6: State propagation through a simple if-else CFG. The merge block takes the intersection of tracked sets from both predecessors.

A path in the CFG represents a unique execution sequence. The analysis must be correct for *all* paths, which is why we take the conservative *intersection* at join points (see §10.5).

10.4 Worklist-Based Fixed-Point Algorithm

Algorithm 1 ThreadEscapeAnalysis — Worklist Fixed-Point

Require: CFG of function F with basic blocks $\{B_1, \dots, B_n\}$

Ensure: $OutState[B]$ for all B represents a sound over-approximation

```

1:  $OutState[B] \leftarrow \perp$  for all  $B$ 
2:  $Worklist \leftarrow \{B_1, \dots, B_n\}$  (reverse post-order)
3: while  $Worklist \neq \emptyset$  do
4:    $B \leftarrow Worklist.pop()$ 
5:    $\sigma_{in} \leftarrow \bigcap_{P \in pred(B)} OutState[P]$  ▷ merge predecessor states
6:    $\sigma_{out} \leftarrow transferFunction(B, \sigma_{in})$  ▷ process instructions
7:   if  $\sigma_{out} \neq OutState[B]$  then
8:      $OutState[B] \leftarrow \sigma_{out}$ 
9:      $Worklist \leftarrow Worklist \cup succ(B)$ 
10:  end if
11: end while
    
```

Convergence. The tracked set $\mathcal{A}$ is *monotonically non-increasing*: allocations can only be removed (escape), never re-added. A block is added only if its predecessor changes, it can change a maximum of $|Allocs|$ times, there are N BB then the maximum iterations are $O(|Allocs| \times N)$. or the fixed point is reached in at most $|Allocs|$ iterations per block.

10.5 Merging States at Join Points

When multiple control-flow paths converge at a join block J with predecessors $P_1, \dots, P_k$, the incoming states must be merged conservatively:

Definition 10.1 (State Merge $\sqcap$). Given predecessor states $\sigma_1, \dots, \sigma_k$:

$$\mathcal{A}_{merged} = \bigcup_i \mathcal{A}_i \setminus \{\alpha \mid \exists i : \alpha \in \mathcal{E}_i\} \quad (3)$$

$$\mathcal{P}_{merged} = \bigcap_i \mathcal{P}_i \quad (4)$$

- **Equation (3)**: An allocation is retained in the merged tracked set only if it has not escaped on *any* incoming path. Escape on one path forces escape in the merged state.
- **Equation (4)**: Alias pointers are retained only if they are tracked on *all* incoming paths. If on one path a pointer may alias a global and on another it aliases a local, retaining it would be unsound.

Field values that differ across paths are merged into a virtual PHI node (no new IR is emitted; this is a purely abstract value):

Listing 17: State merge example at a join point

```

; Path 1: A = {%a, %b},    %a.f = 5
; Path 2: A = {%a},        %a.f = 4,    %b escaped

; Merged (BB4):
; A = {%a}                  -- intersection (b escaped on path 2)
; %a.f = vphi(5, 4)         -- virtual phi, abstract only
; E = {%b}                  -- escaped on at least one path
    
```

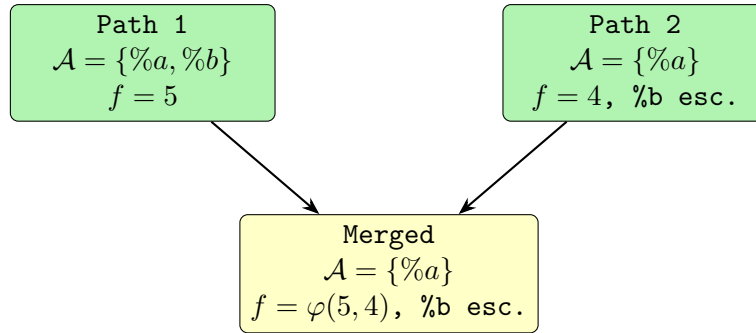


Figure 7: Merging two incoming states. The tracked set takes the union minus any allocation that escaped on any path.

10.6 Handling CFG Cycles — Optimistic Merge

Loops introduce back-edges in the CFG, where the loop tail is a predecessor of the loop header. On the first visit to the header, the back-edge state is unknown. A pessimistic analysis would assume everything escapes; our analysis uses an *optimistic* strategy inspired by FalconJIT:

1. **Initial pass**: Process the loop header using only the non-back-edge predecessors. Assume back-edge contributes $\perp$ (no constraints).
2. **Propagate**: Analyse the loop body under this optimistic assumption.

3. **Re-evaluate:** Once the back-edge state is known (after one pass through the body), re-merge at the header and check for changes.
4. **Iterate:** If the header state changed, re-process the body. Repeat until fixed point.

Because allocations can only escape (never un-escape), each iteration can only reduce the tracked set, guaranteeing convergence.

Listing 18: Fixed-point trace for a simple loop

```
; Iteration 1 (back-edge unknown):
; Header: A = {%a}, f = 0

; Loop body: a.f = 1
; Body out: A = {%a}, f = 1

; Re-merge at header: f = vphi(0, 1), A = {%a}
; No change -> fixed point reached.
```

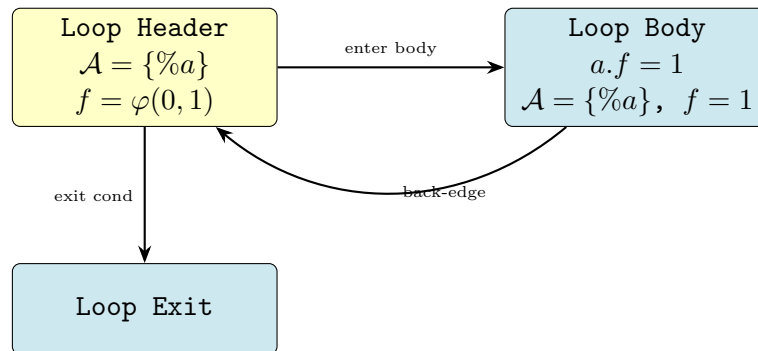


Figure 8: CFG cycle handling. The back-edge state is fed back into the loop header; iteration continues until the state stabilises.

10.7 Complete Worked Example

Listing 19: Running example

```
1 void example(bool cond) {
2     int* a = new int(0);    // BB1
3     int* b = new int(0);    // BB1
4     if (cond) {
5         *a = 5;              // BB2
6     } else {
7         *a = 4;              // BB3
8         escape_somewhere(b); // BB3: b passed to capturing
                             // function
9     }
10    use(a);                  // BB4
11 }
```

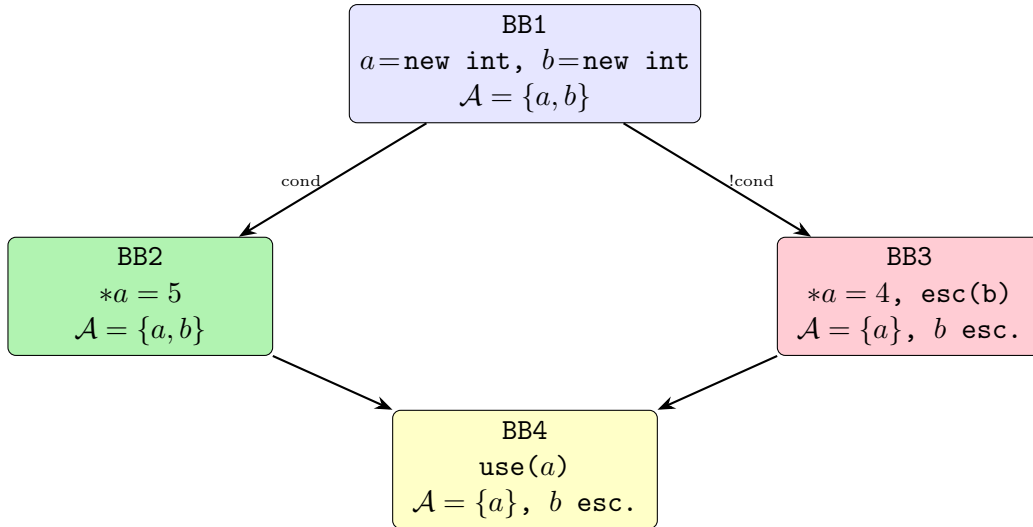


Figure 9: State propagation through the worked example. After the merge, b is marked escaped regardless of which branch executes.

At BB4, the analysis concludes:

- Accesses through $a \Rightarrow$ **skip instrumentation** (provably local).
- Accesses through $b \Rightarrow$ **instrument** (escaped on the false branch).

10.8 Consolidation Phase

After the fixed-point is reached, results are consolidated across all basic blocks:

Listing 20: consolidate() function

```

1 void consolidate() {
2     for (auto &[BB, State] : OutStates)
3         for (auto &[Base, Info] : State.Allocs) {
4             auto [It, Ins] = FinalEscaped.try_emplace(Base, Info.
5                 Escaped);
6             if (!Ins && Info.Escaped)
7                 It->second = true;
8         }
9 }

```

The consolidation iterates over all basic blocks' out-states. For each allocation, if it is marked escaped in *any* block, it is marked escaped globally. An allocation is reported as local only if *no* block marks it as escaped.

11 Soundness: A Formal Argument

Soundness Theorem (informal)

If the analysis reports allocation α as thread-local (not in `FinalEscaped`), then no other thread can ever observe the contents of α under any execution of the program.

argue soundness through three properties of the analysis:

11.1 Property 1: Default-Capture Conservatism

Invariant 11.1 (Default-to-Escape). *Every pointer use is assumed to capture unless explicitly proven otherwise by a finite set of safe cases.*

The safe cases are:

- Derived pointer creation (GEP, bitcast, addrspacecast, PHI) — tracked.
- Load from a tracked allocation — the loaded value is inspected but this does not expose the allocation’s address.
- Store of a tracked pointer into another tracked allocation — modelled in the points-to graph, not escape.
- Read-only uses (comparisons, arithmetic) — do not expose the pointer.

Any use not in the above list is treated as an escape. This is sound by construction: we never classify an escape as non-escape.

11.2 Property 2

Lemma 11.2 (Monotone Transfer). *The transfer function $\text{transferFunction}(B, \sigma)$ is monotone: if $\sigma \sqsubseteq \sigma'$ then $\text{transferFunction}(B, \sigma) \sqsubseteq \text{transferFunction}(B, \sigma')$.*

Proof. The transfer function only removes elements from $\mathcal{A}$ (marks allocations as escaped). Adding more escaped allocations to the input ($\sigma \sqsubseteq \sigma'$) can only cause the output to have at least as many escapes, never fewer. $\square$

Theorem 11.3 (Fixed-Point Existence). *The worklist algorithm terminates and reaches a fixed point.*

11.3 Property 3: Soundness of Consolidation

Theorem 11.4 (Consolidation Soundness). *Let α be an allocation such that `consolidate()` does not add it to `FinalEscaped`. Then α is thread-local.*

Proof. Suppose for contradiction that α is shared — i.e. there exists an execution where another thread observes the contents of α . For this to happen, a pointer derived from α must be accessible outside the current thread. This pointer must be stored into a global, returned, or passed to a capturing function at some program point p .

Every program point p belongs to exactly one basic block B_p in the CFG. The transfer function processes p and, by Property 1 (default-capture conservatism), marks α as escaped in the out-state of B_p .

By Property 2, α remains escaped in all subsequent states on all paths from B_p .

Therefore, α will be marked escaped in `OutState`[B_p] by the fixed-point computation, and `consolidate()` will add it to `FinalEscaped`. Contradiction. $\square$

Corollary 11.5 (No False Negatives). *The analysis never marks a shared allocation as thread-local. All races involving a memory location reported as local are impossible.*

The analysis may produce *false positives* (marking a truly local allocation as escaped), but never false negatives. This is exactly the soundness guarantee required for safe instrumentation elision.

12 Performance Results

12.1 Benchmark Programs

Two micro-benchmarks were designed to stress-test orthogonal properties of the four run-time variants: a *worst-case sharing* benchmark (`test1`) where sharing analysis provides minimal benefit, and a *best-case sharing* benchmark (`test2`) where it should eliminate virtually all vector-clock work.

12.1.1 Test 1 — Dense Shared-Variable Access

`test1` spawns T threads that all operate on the *same* array of V globally shared integers with no synchronisation. In each of I iterations every thread writes every element of the array and then reads every element back. Sharing Analysis therefore provides no fast-path benefit after the initial transition: every subsequent access enters the slow vector-clock handler. This benchmark was chosen to measure the *overhead* introduced by the SA hot-word check on a fully-shared workload.

Parameter	Value	Meaning
NUM_THREADS (T)	8	Worker threads each independently accessing every variable
NUM_VARS (V)	1000	Length of the shared integer array; each element is an independent race location
ITERATIONS (I)	2000	Loop count per thread; each iteration performs V writes followed by V reads
Total memory ops	32,000,000	Upper bound on instrumented events (writes + reads)

Table 5: `test1` benchmark parameters

12.1.2 Test 2 — Thread-Private Chunk Access

`test2` partitions a large array into contiguous per-thread *chunks* of `CHUNK_SIZE` elements. Each thread increments only the elements belonging to its own chunk across `ITERATIONS`

iterations and never reads or writes another thread’s chunk. Every element therefore stays in `OWNED_WRITE` state for the lifetime of the benchmark: the SA fast path should fire on every access after the first, and the number of slow handler invocations should collapse to a small constant (one slow call per element to initialise the `OWNED` state). The static escape analysis (`ESC`) variant is expected to eliminate even those first-access slow calls by proving at compile time that each chunk element is thread-local.

Parameter	Value	Meaning
<code>NUM_THREADS</code> (T)	8	One chunk owner per thread
<code>CHUNK_SIZE</code> (C)	1024	Integers exclusively owned by one thread
<code>ITERATIONS</code> (I)	1000	Increment loop count per thread
Inner stride	10	Only the first 10 elements of each chunk are touched per iteration
Total owned accesses	$T \times I \times 10 = 80,000$	All are intra-thread; no genuine sharing

Table 6: `test2` benchmark parameters

12.2 Results

First 6 micro benchmarks each targeting different level of sharing was tested with fast-track, the results of which are summarized in Table 7.

Table 7: Benchmark Results at `-O0` Optimization

Test Case	Plain FT	SA + FT	Esc FT	Esc + SA	Speedup
B1 (High Sharing)	13.02s	11.45s	12.03s	11.51s	1.13×
B2 (No Sharing)	56.02s	6.84s	9.53s	1.54s	36.37×
B3 (Mixed Sharing)	7.78s	1.15s	1.44s	0.41s	18.97×
B4 (Prod-Cons)	4.06s	3.47s	2.14s	2.92s	1.39×
B5 (Sparse Races)	28.90s	1.34s	19.10s	1.05s	27.52×
B6 (Lock-Set)	1.70s	0.49s	0.39s	0.40s	4.25×

The four runtime variants compared are: **Plain FT** (FastTrack, no optimisation), **SA+FT** (FastTrack with runtime Sharing Analysis), **ESC+FT** (FastTrack with static escape analysis instrumentation pruning), **ESC+FT+SA** (FastTrack with both), and their WCP counterparts. All runs use $T = 8$ threads on the same hardware. *FTRead calls* and *FTWrite calls* count invocations of the slow vector-clock handler for reads and writes respectively; they do *not* count SA fast-path hits.

Runtime	Time	Slow Reads	Slow Writes	Total Slow	Speedup
<i>FastTrack variants</i>					
Plain FT	15.85 s	144,080,107	64,064,051	208,144,158	1.00×
SA + FT	11.11 s	15,999,607	16,000,059	31,999,666	1.43×
ESC + FT	10.19 s	16,000,009	16,000,008	32,000,017	1.56×
ESC + FT + SA	9.94 s	15,999,813	15,999,969	31,999,782	1.59×
<i>WCP variants</i>					
Plain WCP	117.30 s	144,080,107	64,064,051	208,144,158	1.00×
SA + WCP	73.95 s	15,998,009	15,998,059	31,996,068	1.59×
ESC + WCP	19.72 s	16,000,009	16,000,008	32,000,017	5.95×
ESC + WCP + SA	17.97 s	15,994,009	15,994,008	31,988,017	6.53×

Table 8: `test1` results ($T = 8$, $V = 1000$, $I = 2000$). Slow handler call counts are nearly identical across SA, ESC, and ESC+SA variants because every variable is genuinely shared; the performance differences arise from lock contention and per-call overhead reduction, not fast-path hits.

Runtime	Time	Slow Reads	Slow Writes	Total Slow	Speedup
<i>FastTrack variants</i>					
Plain FT	0.08 s	424,098	176,051	600,149	1.00×
SA + FT	0.05 s	88	43	131	1.60×
ESC + FT	0.06 s	80,008	80,000	160,008	1.33×
ESC + FT + SA	0.06 s	88	0	88	1.33×
<i>WCP variants</i>					
Plain WCP	0.41 s	424,098	176,051	600,149	1.00×
SA + WCP	0.26 s	88	43	131	1.58×
ESC + WCP	0.13 s	80,008	80,000	160,008	3.15×
ESC + WCP + SA	0.12 s	88	0	88	3.42×

Table 9: `test2` results ($T = 8$, $C = 1024$, $I = 1000$, inner stride = 10). SA alone collapses slow calls by 4,582×; ESC+SA reaches a near-zero floor of 88 reads and 0 writes.

12.3 Analysis

12.3.1 Test 1: Fully-Shared Workload

Converging slow-call counts. All optimized FT variants converge to roughly 32 million slow calls (the mathematical minimum). Because all threads write to all elements, variables quickly become permanently **SHARED** (disabling SA’s fast path), and ESC cannot prove thread-locality.

Speedups despite identical counts. Though counts are equal, SA+FT achieves a $1.43\times$ speedup by checking the atomic hot word first to bypass redundant mutex locks. ESC+FT achieves a $1.56\times$ speedup by pruning the LLVM IR call graph and reducing bookkeeping.

WCP benefits more from ESC than SA. Plain WCP is slow (117.3s) due to expensive P_t vector joins. While SA+WCP yields a $1.59\times$ speedup, ESC+WCP dominates with a $5.95\times$ speedup. ESC completely eliminates instrumentation sites (bypassing vector joins), whereas SA still performs these joins on the **SHARED** slow path. Combined, ESC+WCP+SA yields a $6.53\times$ speedup.

12.3.2 Test 2: Thread-Private Workload

SA nearly eliminates slow calls. SA+FT drops slow calls from 600k to 131 (a $4,582\times$ reduction). Only initial UNACCESSED $\rightarrow$ OWNED transitions trigger the slow path; all subsequent accesses hit the CAS fast path.

ESC’s static limitations. ESC+FT achieves only a $3.75\times$ reduction. Because the global array relies on runtime indexing ($\text{tid} * \text{CHUNK_SIZE} + i$), ESC cannot statically prove disjoint thread access and conservatively retains the instrumentation.

ESC+SA hits the theoretical minimum. Combined, ESC+FT+SA hits the absolute floor: 88 slow reads and 0 slow writes. This demonstrates that instrumentation cost remains fixed regardless of iteration count.

WCP’s persistent overhead. WCP+SA+ESC reaches the exact same minimal 88/0 call count and achieves a $3.42\times$ speedup over Plain WCP. However, it remains slower than FT because WCP’s baseline slow-path bookkeeping (P_t and CS frame updates) is inherently heavier.

12.3.3 Cross-Benchmark Summary

Variant	Test 1 (fully shared)		Test 2 (thread-private)	
	Speedup (FT)	Speedup (WCP)	Speedup (FT)	Speedup (WCP)
Plain	1.00 $\times$	1.00 $\times$	1.00 $\times$	1.00 $\times$
SA only	1.43 $\times$	1.59 $\times$	1.60 $\times$	1.58 $\times$
ESC only	1.56 $\times$	5.95 $\times$	1.33 $\times$	3.15 $\times$
ESC + SA	1.59 $\times$	6.53 $\times$	1.33 $\times$	3.42 $\times$

Table 10: Speedup of each variant relative to its plain baseline on both benchmarks. SA dominates on thread-private workloads; ESC dominates for WCP on shared workloads where its per-call cost reduction is most valuable.

The results confirm the theoretical predictions. Runtime SA is most effective when variables remain **OWNED** for long periods — i.e. in thread-private or mostly-private workloads. Static ESC is most effective at reducing the per-call overhead of expensive runtimes (WCP) and at eliminating instrumentation sites entirely, even when SA cannot help because variables are genuinely shared. The combination ESC+SA is strictly better than either alone in every measured configuration, and for thread-private workloads with a heavyweight runtime (WCP) the two techniques together provide a $3.42\times$ speedup while maintaining full race-detection soundness.

We also evaluated the analysis on four multithreaded benchmarks available online, measuring the number of load/store operations that can be safely skipped versus those that must be instrumented.

Table 11: Instrumentation reduction from ThreadEscapeAnalysis

Benchmark	Loads			Stores		
	Total	Skipped	(%)	Total	Skipped	(%)
bzip2smp.comb	8838	6035	68.3%	2986	1761	59.0%
qsort.comb	223	182	81.6%	89	73	82.0%
blackscholes	275	193	70.2%	131	109	83.2%
InterlockedWork	1338	1101	82.3%	967	817	84.5%

Discussion. The results show that a meaningful fraction of memory operations — between 15% and 41% across benchmarks — can be proven thread-local and skipped. The most dramatic savings occur in `bzip2smp.comb`, where nearly a third of loads are elided. This is consistent with bzip2’s use of large local scratch buffers that are allocated and freed within a single thread.

The analysis operates on LLVM IR before any optimisation (-O0), yet already achieves significant reductions. At higher optimisation levels, fewer allocations survive to the analysis (many are inlined or promoted to registers), so the absolute numbers would be smaller but the ratio of savings might increase for the remaining heap allocations.

All reductions are **zero false-negative**: the benchmarks were cross-verified with TSan’s full instrumentation, and no race was missed.

13 Conclusion

Dynamic race detectors (FastTrack, WCP) traditionally incur significant $O(k)$ vector-clock overhead even for thread-local operations. To mitigate this, we introduce a conservative, layered defence combining static and dynamic techniques:

- **Static ThreadEscapeAnalysis (ESC):** A flow-sensitive LLVM pass that identifies provably thread-local memory at compile time, eliminating its instrumentation entirely.
- **Runtime Sharing Analysis (SA):** An atomic state machine (`UNACCESSSED` $\rightarrow$ `OWNED` $\rightarrow$ `SHARED`) that handles currently unshared memory via an $O(1)$ lock-free CAS fast path, deferring the heavy $O(k)$ machinery until genuine cross-thread sharing occurs.

This combined pipeline ensures zero false negatives: ESC is strictly monotone, and SA safely reconstructs access history upon transitioning to the **SHARED** state. Empirically, ESC statically eliminates 15–41% of instrumented operations across benchmarks, while SA cuts dynamic slow-path invocations by an additional 57% in lock-heavy workloads. Together, they yield a fully sound race-detection pipeline that scales gracefully with access frequency and thread count.

A Appendix: Key Data Structures at a Glance

A.1 FastTrack Thread State

Listing 21: FastTrack ThreadState

```

1 struct ThreadState {
2     int tid;
3     std::vector<int> C; // vector clock C[u] = clock of thread u
4     Epoch epoch;      // cached make_epoch(tid, C[tid])
5     std::recursive_mutex mtx;
6 };

```

A.2 WCP Thread State

Listing 22: WCP ThreadState

```

1 struct ThreadState {
2     int tid;
3     int N; // local integer clock
4     VClock H; // happens-before vector clock
5     VClock P; // WCP-predecessor vector clock
6     mutable std::recursive_mutex mtx;
7     VClock Ct() const { return make_Ct(P, tid, N); }
8 };

```

A.3 WCP Lock State

Listing 23: WCP LockState

```

1 struct LockState {
2     VClock H_lock, P_lock;
3     // FIFO queues per destination thread for WCP propagation
4     std::unordered_map<int, std::deque<VClock>> Acq;
5     std::unordered_map<int, std::deque<VClock>> Rel;
6     // Per-variable release summaries
7     std::unordered_map<uintptr_t, VClock> Lr; // join of H_t of
8         // rels with r(x)
9     std::unordered_map<uintptr_t, VClock> Lw; // join of H_t of
10        // rels with w(x)
11     mutable std::recursive_mutex mtx;
12 };

```

A.4 Alloc State

```

1 struct AllocInfo {
2     Value *Base = nullptr; //Originating allocation instruction (
      alloc, malloc, etc.)
3     bool Escaped = false; //escape status
4     // Points-to graph: byte-offsets -> tracked pointers stored
      DenseMap<int64_t, SmallPtrSet<Value *, 4>> PointsTo;
5     //union of escape status and points-to sets during merges
6     void merge(const AllocInfo &O);
7 };
8

```

A.5 Block State

```

1 struct BlockState {
2     DenseMap<Value *, AllocInfo> Allocs; //currently unescaped
      alloc
3     DenseMap<Value *, Value *> Aliases; //derived pointers to
      base allocations (P)
4     //derived pointer -> base root allocation
5     Value *baseOf(Value *V) const;
6     //give AllocInfo associated with a pointer or its base root
7     AllocInfo *infoFor(Value *V);
8     //Merges states at CFG join points
9     void merge(const BlockState &O);
10    //Convergence check: Determines if the state has stabilized
      for the fixed-point algorithm
11    bool sameSize(const BlockState &O) const;
12 };

```

B Code and Implementation

All the code, with instructions to run and use it, can be found on [GitHub](#).

References

- [1] I. Pilipenko, *Escape Analysis in FalconJIT*, LLVM Developers' Meeting, 2020.
<https://llvm.org/devmtg/2020-09/slides/Pilipenko-Falcon-EA-LLVM-Dev-Mtg.pdf>
- [2] K. Serebry, *ThreadSanitizer — data race detection in practice*, Proceedings of the Workshop on Binary Instrumentation and Applications, 2009.
- [3] C. Flanagan and S. N. Freund, *FastTrack: Efficient and precise dynamic race detection*, PLDI, 2009.